

UNIVERSIDAD TECNOLÓGICA NACIONAL
Tecnicatura Universitaria en Programación a Distancia

TRABAJO PRÁCTICO N.º 1

Alumna: Sabrina Gimenez

Comisión 4

Del dominio al esquema: modelo entidad-relación, modelo relacional, normalización y DDL

Materia	Base de Datos I
Carrera	Tecnicatura Universitaria en Programación a Distancia (UTN)
Trabajo práctico	N.º 1 — Del dominio al esquema: modelo ER, modelo relacional, normalización y DDL
Proyecto integrador	Food Store (sistema de gestión de pedidos de un negocio de comidas) — pieza correspondiente a la Semana 1

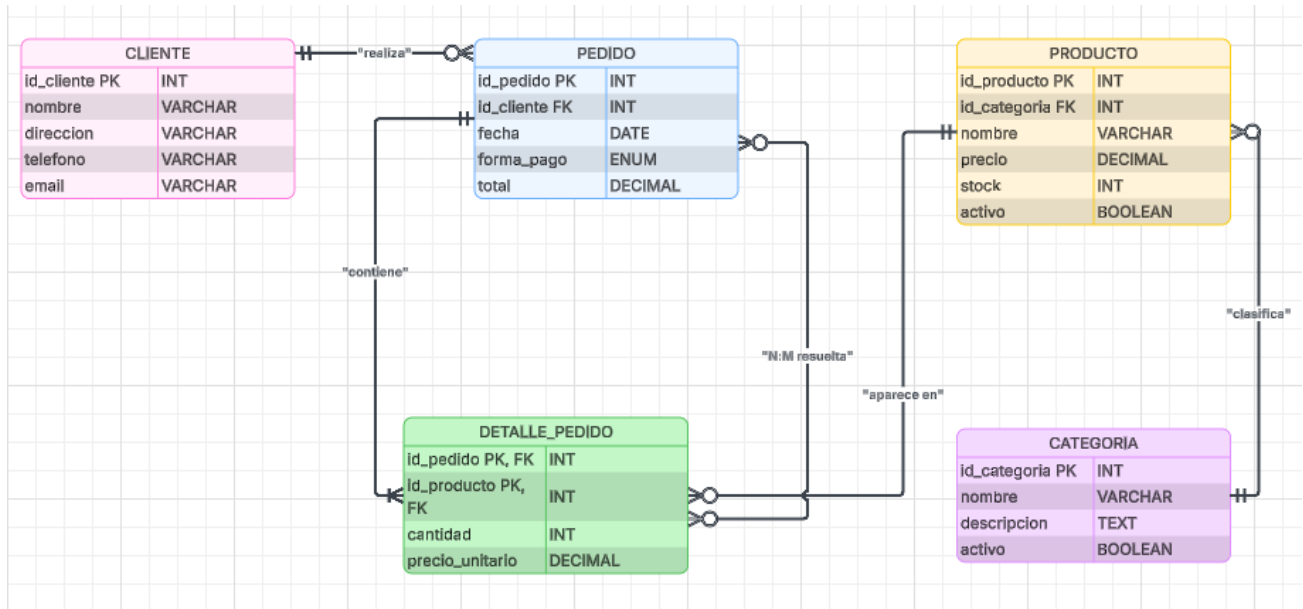
Presentación y objetivos

Este trabajo práctico inaugura el hilo conductor de la materia: el proyecto integrador **Food Store**, un sistema de gestión de pedidos para un negocio de comidas. A partir de un enunciado en lenguaje natural, van a recorrer el camino completo que hace una base de datos relacional bien diseñada: analizar el dominio, modelarlo con un diagrama entidad-relación, derivar el esquema relacional correspondiente, verificar (y justificar) que ese esquema está normalizado, y finalmente escribir el DDL que lo crea en PostgreSQL.

Al finalizar este trabajo van a haber producido **schema.sql**, la primera pieza del proyecto integrador, que se irá ampliando semana a semana con datos, consultas, vistas, funciones y transacciones.

Parte 1 — Modelo entidad-relación (25%)

Diagrama ER



Diccionario

Entidades principales

- **Cliente**
 - PK: id_cliente
 - Atributos: nombre, dirección, teléfono, email (único)
 - Participación: todo pedido pertenece a un cliente → participación **total** en la relación con Pedido.
- **Producto**
 - PK: id_producto
 - Atributos: nombre, precio, stock, activo (booleano)
 - Participación: cada producto pertenece a una categoría → participación **total** en la relación con Categoría.
- **Categoría**
 - PK: id_categoria
 - Atributos: nombre, descripción, activo
 - Participación: puede existir sin productos → participación **parcial**.
- **Pedido**
 - PK: id_pedido
 - Atributos: fecha, forma_pago (ENUM), total
 - Relación: pertenece a un cliente.
- **DetallePedido** (tabla intermedia)
 - PK: (id_pedido, id_producto) o surrogate key
 - Atributos: cantidad, precio_unitario
 - Relación: N:M entre Pedido y Producto.

Relaciones

- Cliente — Pedido: **1:N** (un cliente puede tener muchos pedidos).
- Categoría — Producto: **1:N** (una categoría puede tener muchos productos).
- Pedido — Producto: **N:M** resuelta con DetallePedido.
- Pedido — DetallePedido: **1:N**.
- Producto — DetallePedido: **1:N**.

Preguntas guía

1. ¿Por qué la relación entre producto y pedido no puede resolverse como una relación binaria simple 1:N? ¿Qué información se perdería si lo intentaras?

Porque perderías los atributos propios de la relación (cantidad y precio_unitario).

2. ¿Qué entidad tiene participación parcial en la relación con categoría, y cuál tiene participación total? ¿Qué pasaría si invirtieras esa lectura?

Categoría → parcial, Producto → total. Si lo inviertes, obligarías a que cada categoría tenga al menos un producto, lo cual contradice la regla R1: Todo producto pertenece exactamente a una categoría; una categoría puede tener muchos productos, o ninguno todavía (por ejemplo, una categoría recién creada).

3. ¿Alguno de tus atributos podría descomponerse en partes más simples (por ejemplo, un nombre completo en nombre y apellido)? Justificá tu decisión de dejarlo compuesto o de separarlo.

Nombre completo del cliente, podrías descomponerlo en nombre y apellido para mayor flexibilidad.

Parte 2 — Derivación al modelo relacional (20%)

Tablas derivadas

Tabla	Clave primaria	Claves foráneas	Atributos
<u>cliente</u>	id_cliente	No hay	nombre, direccion, telefono, email, activo
<u>categoria</u>	id_categoria	No hay	nombre, descripcion, activo
<u>producto</u>	id_producto	categoria_id → categoria	nombre, precio, stock, activo
<u>pedido</u>	id_pedido	cliente_id → cliente	fecha, forma_pago, total
<u>detalle_pedido</u>	(id_pedido, id_producto)	id_pedido → pedido, id_producto → producto	cantidad, precio_unitario

Relaciones

- **Cliente → Pedido (1:N):** Cada pedido pertenece a un cliente, pero un cliente puede tener varios pedidos. → Se agrega `cliente_id` como FK en `pedido`.

- **Categoría → Producto (1:N):** Cada producto pertenece a una categoría, pero una categoría puede tener muchos productos o ninguno. → Se agrega `categoria_id` como FK en `producto`.
- **Pedido ↔ Producto (N:M):** Un pedido puede incluir varios productos y un producto puede aparecer en muchos pedidos. → Se crea la tabla intermedia `detalle_pedido` con las FKs `id_pedido` y `id_producto`.

Preguntas guía:

- ¿Qué pasaría con la integridad de tus datos si la tabla intermedia no incluyera ambas claves foráneas como NOT NULL?
Se perdería la integridad referencial: podrían existir líneas de detalle sin pedido o sin producto.
- Si el pedido pudiera existir sin ningún producto asociado (por ejemplo, un pedido recién iniciado), ¿cambia esto la participación que definiste en la Parte 1 para la relación con la tabla intermedia? Explica el impacto.
La participación del pedido en la relación con `detalle_pedido` es **parcial**. Esto no cambia la estructura del modelo, pero implica que `detalle_pedido` puede estar vacío para un pedido recién iniciado.

Notación

cliente (`id_cliente`, nombre, direccion, telefono, email, activo)

categoria (`id_categoria`, nombre, descripcion, activo)

producto (`id_producto`, nombre, precio, stock, activo, `categoria_id` -> categoria)

pedido (`id_pedido`, fecha, forma_pago, total, `cliente_id` -> cliente)

detalle_pedido (`id_pedido` -> pedido, `id_producto` -> producto, cantidad, precio_unitario)

Parte 3 — Normalización hasta 3FN/BCNF (25%)

Food Store todavía registra sus ventas en una única planilla plana. Este es un extracto real de esa planilla (una fila por cada línea de producto vendido dentro de un pedido):

N.º pedido	Fecha	Cliente	Producto	Categoría	Precio unitario	Cant.	Subtotal	Forma de pago
1	01/03/2026	Ana Gómez	Muzzarella	Pizzas	1000.00	2	2000.00	EFFECTIVO
1	01/03/2026	Ana Gómez	Coca 1.5L	Bebidas	800.00	1	800.00	EFFECTIVO
2	01/03/2026	Luis Paz	Napolitana	Pizzas	1500.00	1	1500.00	TARJETA
3	05/03/2026	Ana Gómez	Muzzarella	Pizzas	1050.00	3	3150.00	TRANSFERENCIA
4	06/03/2026	Marta Ruiz	Coca 1.5L	Bebidas	800.00	4	3200.00	EFFECTIVO
4	06/03/2026	Marta Ruiz	Napolitana	Pizzas	1500.00	2	3000.00	EFFECTIVO
5	07/03/2026	Luis Paz	Muzzarella	Pizzas	1050.00	1	1050.00	TARJETA

Paso 1: Clave candidata

Cada fila de la planilla representa un producto dentro de un pedido. La clave que identifica una fila es (**nro_pedido, producto**). Esto evita duplicados porque dentro de un pedido un producto no se repite.

Paso 2: Dependencias funcionales

- nro_pedido → fecha, cliente, forma_pago
- producto → categoría
- (nro_pedido, producto) → precio_unitario, cantidad, subtotal
- precio_unitario × cantidad → subtotal (dependencia derivada)

Paso 3: Verificación de 1FN

La planilla ya está en 1FN: todos los atributos son atómicos, sin listas ni grupos repetidos.

Paso 4: Verificación de 2FN

La clave es compuesta (nro_pedido, producto).

- fecha, cliente, forma_pago dependen solo de nro_pedido → **dependencia parcial**.
- categoría depende solo de producto → **dependencia parcial**.

Paso 5: Verificación de 3FN

Después de separar:

- cliente → correo electrónico (clave candidata en el modelo ER).
- categoría depende de producto, pero ya está en su tabla. No quedan dependencias transitivas.

Tablas resultantes

1. Clientes

id_cliente	nombre	email (único)
1	Ana Gómez	ana@mail.com
2	Luis Paz	luis@mail.com
3	Marta Ruiz	marta@mail.com

2. Categoría

id_categoria	nombre
1	Pizzas
2	Bebidas

3. Producto

id_producto	nombre	categoria_id
1	Muzzarella	1
2	Napolitana	1
3	Coca 1.5L	2

4. Pedido

id_pedido	fecha	cliente_id	forma_pago
1	01/03/2026	1	EFFECTIVO
2	01/03/2026	2	TARJETA
3	05/03/2026	1	TRANSFERENCIA
4	06/03/2026	3	EFFECTIVO
5	07/03/2026	2	TARJETA

5. DetallePedido

id_pedido	id_producto	cantidad	precio_unitario	subtotal
1	1	2	1000.00	2000.00
1	3	1	800.00	800.00
2	2	1	1500.00	1500.00
3	1	3	1050.00	3150.00
4	3	4	800.00	3200.00
4	2	2	1500.00	3000.00
5	1	1	1050.00	1050.00

Paso 6: Verificación de BCNF

En cada tabla, todo determinante es clave candidata. → Sí, todas cumplen BCNF.

Tablas normalizadas finales

1. **Pedido** (nro_pedido PK, fecha, cliente, forma_pago)
2. **Cliente** (cliente PK, email UNIQUE, otros datos)
3. **Producto** (producto PK, categoria, activo, precio_lista)
4. **Categoria** (categoria PK, descripcion)
5. **DetallePedido** (nro_pedido FK, producto FK, cantidad, precio_unitario, subtotal opcional)

Preguntas de integración

- Comparé las tablas que obtuviste normalizando la planilla con las tablas que derivaste en la Parte 2 a partir de tu modelo ER. ¿A qué entidad de tu ER corresponde cada tabla? ¿Hay atributos de tu ER (por ejemplo, el mail del cliente) que no aparecían en esta planilla? ¿Qué te dice eso sobre la diferencia entre modelar a partir de un enunciado de negocio y normalizar a partir de datos históricos ya existentes?

Las tablas coinciden: cliente, producto, categoría, pedido y detalle_pedido. Diferencia: en la planilla no aparece el email del cliente, pero sí en el modelo ER. → Esto muestra que modelar desde reglas de negocio es más completo que normalizar datos históricos.

- El atributo subtotal de cada línea es, en teoría, calculable como cantidad × precio_unitario. ¿Conviene almacenarlo igual, o siempre recalcularlo al vuelo? ¿Esa decisión de diseño viola alguna forma normal, o es un tema distinto (rendimiento, trazabilidad histórica)? Justificá en dos o tres líneas.

No viola ninguna forma normal porque es un atributo derivado. Decisión de diseño: se puede recalcular al vuelo (rendimiento bajo) o almacenar para acelerar consultas y trazabilidad histórica.

Parte 4 — DDL en PostgreSQL (25%)

Con este prompt, OpenCode generó mi archivo `schema.sql` completo y listo para aplicar:

Genera un archivo `schema.sql` en PostgreSQL para el sistema Food Store con las siguientes especificaciones:

1. Tablas:

- cliente (id_cliente PK autoincremental, nombre, direccion, telefono, email UNIQUE, activo BOOLEAN con DEFAULT TRUE, created_at TIMESTAMPTZ con DEFAULT now()).
- categoria (id_categoria PK autoincremental, nombre UNIQUE, descripcion, activo BOOLEAN con DEFAULT TRUE).
- producto (id_producto PK autoincremental, nombre, precio NUMERIC(10,2) CHECK >=0, stock INT CHECK >=0, activo BOOLEAN DEFAULT TRUE, categoria_id FK -> categoria ON DELETE RESTRICT).
- pedido (id_pedido PK autoincremental, fecha TIMESTAMPTZ DEFAULT now(), forma_pago ENUM con valores ('EFFECTIVO','TARJETA','TRANSFERENCIA'), total NUMERIC(12,2) CHECK >=0, cliente_id FK -> cliente ON DELETE RESTRICT).
- detalle_pedido (id_detalle PK autoincremental, id_pedido FK -> pedido ON DELETE CASCADE, id_producto FK -> producto ON DELETE RESTRICT, cantidad INT CHECK >0, precio_unitario NUMERIC(10,2) CHECK >=0, subtotal NUMERIC(12,2) generado como cantidad * precio_unitario).

2. Restricciones:

- Todas las claves primarias explícitas.
- Claves foráneas con REFERENCES y comportamiento ON DELETE justificado.
- Al menos una restricción UNIQUE (email en cliente, nombre en categoría).
- Al menos tres restricciones CHECK (precio ≥ 0 , stock ≥ 0 , cantidad > 0).
- NOT NULL en atributos de participación total.
- DEFAULT en atributos como activo y fecha.

3. Índices:

- Índice para acelerar búsqueda de pedidos por cliente.
- Índice para listar productos activos por categoría.

4. Comentarios:

- Incluye comentarios breves en el script explicando decisiones de diseño (ejemplo: por qué ON DELETE RESTRICT en producto $\rightarrow$ categoría).

El archivo debe ser ejecutable sin errores en PostgreSQL.

Conclusión personal

Este trabajo práctico me resultó sumamente integrador, ya que me permitió repasar en profundidad los conocimientos adquiridos en **Base de Datos I** y conectarlos con las nuevas herramientas disponibles en la actualidad. La experiencia de diseñar **diagramas ER** y elaborar **prompts** útiles para guiar la generación de resultados me demostró que es posible lograr un proceso más eficiente y rápido, sin perder rigurosidad en el análisis ni en la implementación.